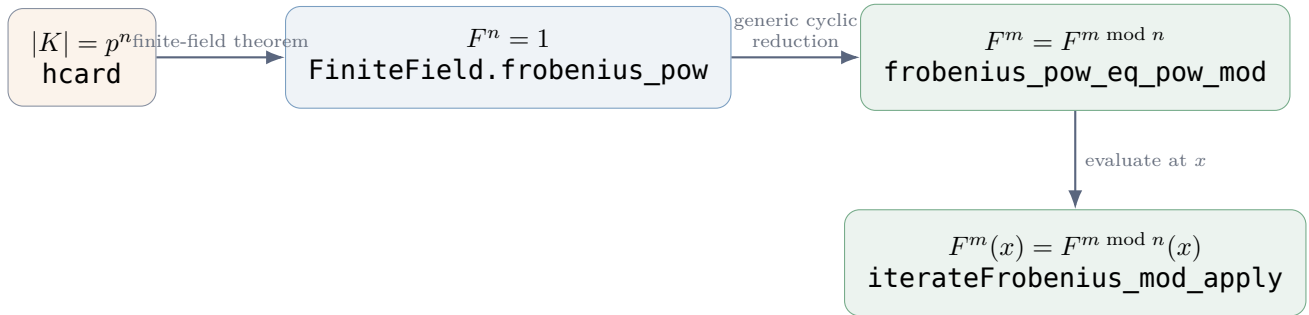


Frobenius Periodicity as an Explicit Normalization API

A visual guide to orientation, naming, placement, and use in Lean 4

Finite fields K with $|K| = p^n$



Recommended declarations

```

theorem frobenius_pow_eq_pow_mod
  (hcard : Fintype.card K = p ^ n) (m : Nat) :
  frobenius K p ^ m =
    frobenius K p ^ (m % n) :=
  pow_eq_pow_mod m (FiniteField.frobenius_pow hcard)

theorem iterateFrobenius_mod_apply
  (hcard : Fintype.card K = p ^ n)
  (m : Nat) (x : K) :
  iterateFrobenius K p m x =
    iterateFrobenius K p (m % n) x := by
  rw [iterateFrobenius_eq_pow,
    iterateFrobenius_eq_pow,
    frobenius_pow_eq_pow_mod hcard m]
  
```

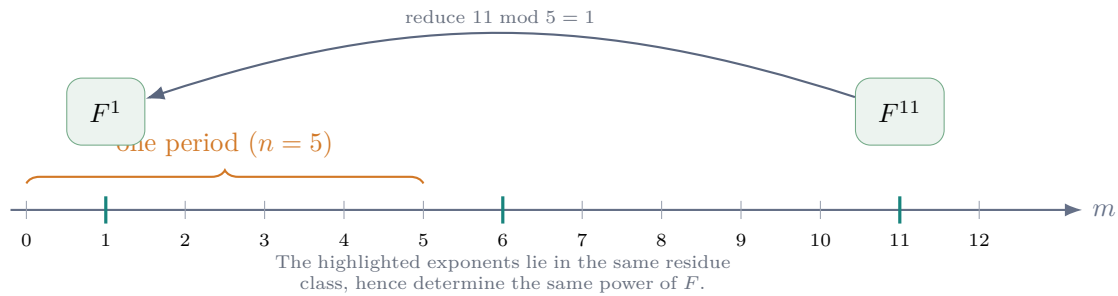
The four API decisions

Question	Decision
Which orientation?	unrestricted exponent on the left
Global simplification?	no <code>[simp]</code> attribute
Where to place it?	after <code>FiniteField.frobenius_pow</code>
Pointwise name?	<code>iterateFrobenius_mod_apply</code>

Core idea. Syntactic size and mathematical complexity are different. Although `m % n` is a larger term, it is the canonical representative of the exponent modulo the period.

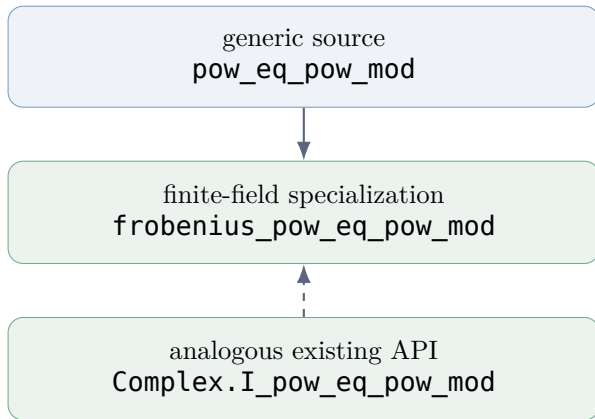
1. Why the “larger” right-hand side is the normal form

Let $F = \text{Frob}_p$. Since $F^n = 1$, the value of F^m depends only on the residue class of m modulo n . Normalization chooses the canonical representative $m \bmod n$.



Three aligned precedents

The recommended orientation is not an isolated style preference. It follows the whole theorem chain:

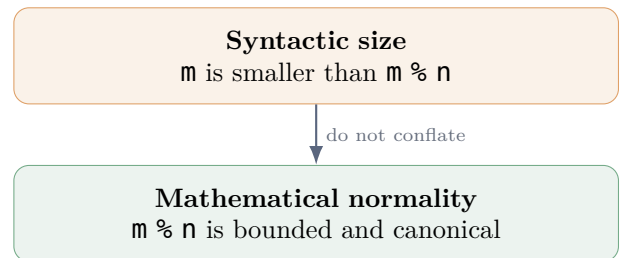


All three read

arbitrary power = reduced power.

The wrapper therefore preserves the direction of its generic source theorem.

Two notions of complexity



The goal of this theorem is periodic normalization, not raw AST-size reduction. The right-hand side:

- records the period explicitly;
- bounds the exponent by n when $n > 0$;
- makes equivalent iterates visibly comparable;
- exposes a canonical representative for later proofs.

Not recommended as the declaration orientation:

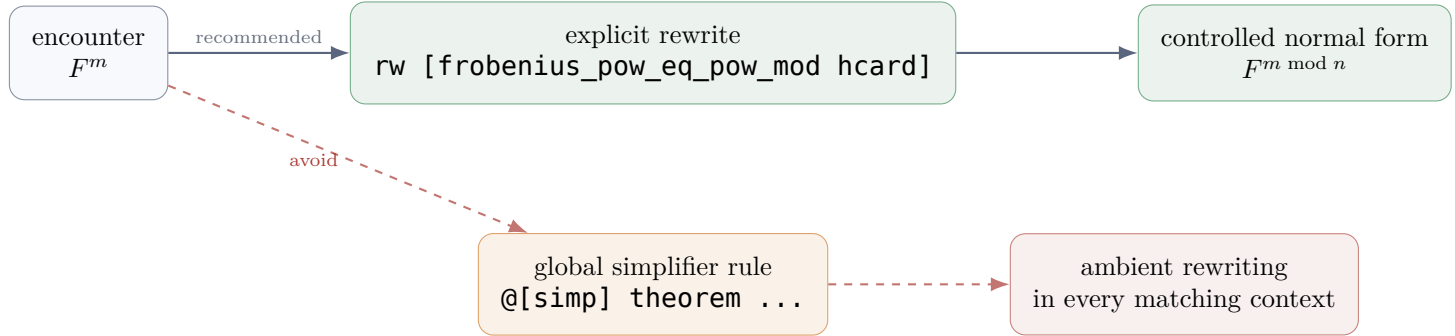
$$F^{m \bmod n} = F^m.$$

It reads as expansion and points opposite to the source theorem.

Rule of thumb: orient an explicit normalization lemma toward the canonical mathematical representative, even when that representative has a larger printed term.

2. Explicit rewrite, not global simplification

The orientation answers “what is the normal form?” The `[simp]` decision answers a different question: “should every simplifier invocation perform this normalization automatically?”



Why explicit rewriting fits

- The cardinality witness `hcard` is meaningful local data.
- Users decide exactly where exponent reduction is useful.
- Proof scripts advertise the normalization step.
- The theorem remains available in either direction via `rw` or symmetry.
- The term-size concern stays local rather than becoming simplifier policy.

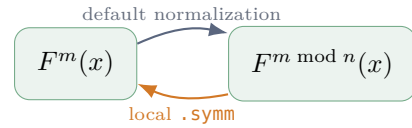
```
-- Hom-level normalization
rw [FiniteField.frobenius_pow_eq_pow_mod hcard]

-- Pointwise normalization
rw [FiniteField.iterateFrobenius_mod_apply hcard]
```

Orientation does not forbid reverse use

A theorem has a preferred reading direction, but equality remains symmetric. Expand only at the call site that needs it:

```
exact
(FiniteField.iterateFrobenius_mod_apply
 hcard m x).symm
```



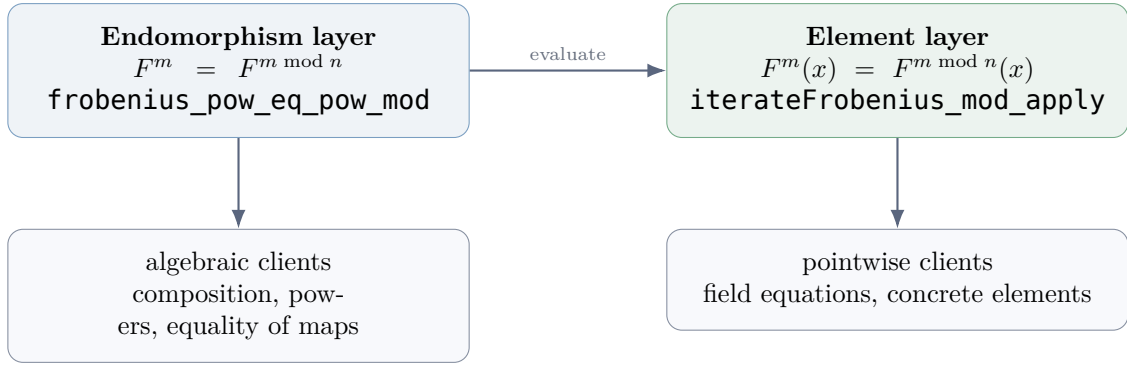
Occasional reverse use is not evidence for reversing the declaration. API orientation should represent the dominant mathematical normalization.

Decision table

Situation	Tool	Why
Normalize one hom expression	<code>rw [frobenius_pow_eq_pow_mod hcard]</code>	direct algebraic equality
Normalize one element expression	<code>rw [iterateFrobenius_mod_apply hcard]</code>	hides ring-hom powers
Need the unreduced form	theorem with <code>.symm</code> , or <code>rw [← ...]</code>	explicit, exceptional expansion
Routine unrelated simplification	leave these theorems unused	avoids global policy and term churn

3. Two API layers for two kinds of clients

The hom-level and pointwise statements express the same mathematics, but they serve different proof interfaces.

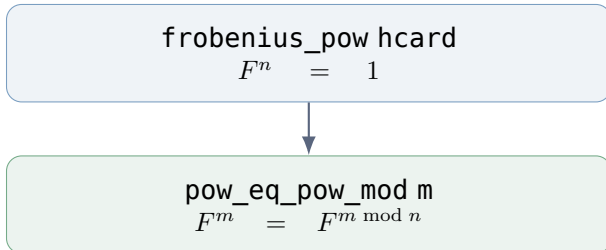


Hom-level example

```
example (hcard : Fintype.card K = p ^ n)
  (m : Nat) :
    frobenius K p ^ (m + n) =
      frobenius K p ^ ((m + n) % n) := by
      rw [FiniteField.frobenius_pow_eq_pow_mod hcard]
```

Use this form when the surrounding goal already treats Frobenius as a ring endomorphism. It preserves the structural level and can be transported or composed as an equality of maps.

Minimal proof architecture



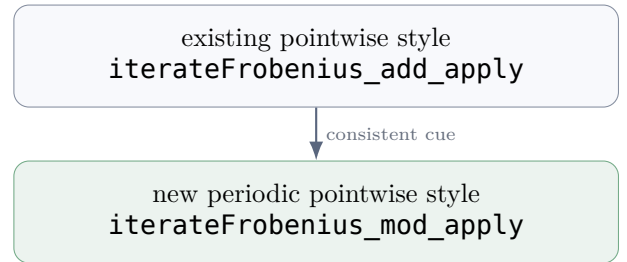
The first declaration is a direct generic specialization, so it belongs immediately beside its source theorem.

Pointwise example

```
example (hcard : Fintype.card K = p ^ n)
  (m : Nat) (x : K) :
    iterateFrobenius K p (m + n) x =
      iterateFrobenius K p ((m + n) % n) x := by
      rw [FiniteField.iterateFrobenius_mod_apply hcard]
```

This wrapper keeps implementation details out of element-level proofs: clients need not manually expose `iterateFrobenius` as a power of a ring hom.

Why the name ends in `_apply`

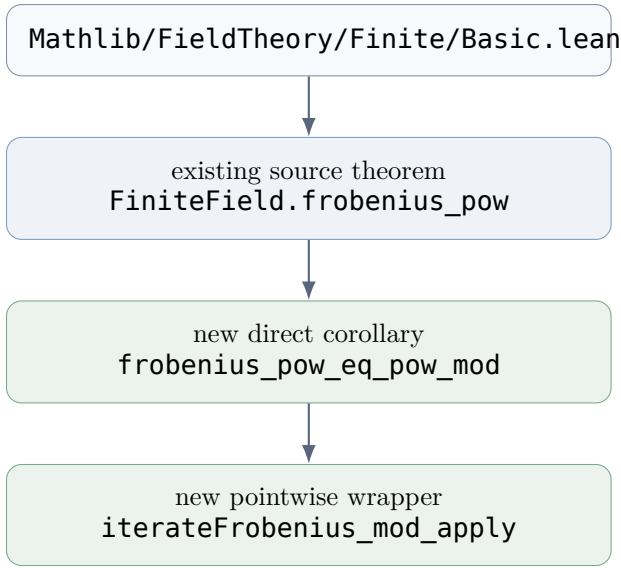


The suffix tells readers that the theorem concludes an equality after application to an element, distinguishing it at the call site from the ring-hom equality.

API principle: expose the structural theorem and a thin pointwise wrapper. The wrapper is not redundant when it removes representation details from a common client layer.

4. Placement, edge cases, and review checklist

Recommended source order



This order keeps proof dependencies visible. No additional finite-field machinery is required beyond the existing basic module.

The $n = 0$ edge case

No hypothesis $0 < n$ is needed:

$$m \bmod 0 = m,$$

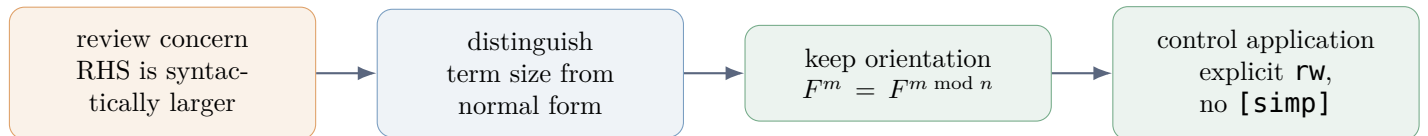
so the conclusion becomes $F^m = F^m$. Moreover, in the intended field setting the premise $|K| = p^0 = 1$ cannot describe a field's cardinality. The statement is already total and correct without an artificial positivity side condition.

What the final API says

Choice	Verdict
Unrestricted exponent on left	YES
Modulo exponent on right	YES
Match generic theorem direction	YES
Provide hom-level theorem	YES
Provide pointwise wrapper	YES
Use <code>_apply</code> pointwise suffix	YES
Place after <code>frobenius_pow</code>	YES
Add $0 < n$	NO
Mark either theorem <code>[simp]</code>	NO
Reverse declaration by default	NO

Submission checklist

- ☐ Preserve theorem orientation.
- ☐ Keep both declarations free of `[simp]`.
- ☐ Put the direct corollary beside `frobenius_pow`.
- ☐ Use the pointwise `_mod_apply` suffix.
- ☐ Include endomorphism, pointwise, and reverse-use examples.
- ☐ Build the target and inspect theorem axioms.



Final recommendation

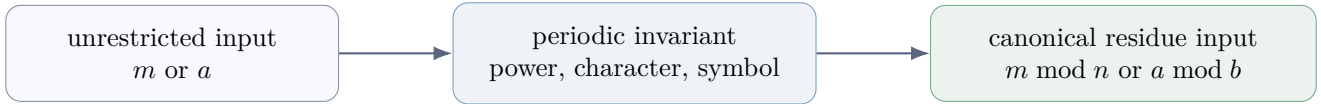
Keep the original equality orientation because modulo reduction is the intended mathematical normal form and matches both the generic source theorem and analogous API. Keep normalization explicit rather than global. Place the hom theorem immediately after its source, and name the pointwise wrapper `iterateFrobenius_mod_apply`.

The corresponding Lean declarations and all three usage modes are provided in `MathlibExtras/FieldTheory/Finite/FrobeniusPeriod.lean` and `MathlibExtras/FieldTheory/Finite/FrobeniusPeriodExamples.lean`.

Appendix A. What the Lean and Mathlib sources actually do

A regular-expression survey of Lean 4.28.0 and the checked-out Mathlib revision found a broad family of declarations that deliberately place a reduced residue expression on the right. These are stronger precedents than the complex-number example alone.

Declaration	Equality shape	[simp]?
<code>pow_eq_pow_mod</code>	$a^m = a^{m \bmod n}$	no
<code>Complex.I_pow_eq_pow_mod</code>	$I^m = I^{m \bmod 4}$	no
<code>Int.units_pow_eq_pow_mod_two</code>	$u^m = u^{m \bmod 2}$	no
<code>neg_one_pow_eq_pow_mod_two</code>	$(-1)^m = (-1)^{m \bmod 2}$	no
<code>χ_4_nat_mod_four</code>	$\chi_4(m) = \chi_4(m \bmod 4)$	no
<code>χ_8_nat_mod_eight</code>	$\chi_8(m) = \chi_8(m \bmod 8)$	no
<code>legendreSym.mod</code>	$\left(\frac{a}{p}\right) = \left(\frac{a \bmod p}{p}\right)$	no
<code>jacobiSym.mod_left</code>	$J(a \mid b) = J(a \bmod b \mid b)$	no
<code>jacobiSym.mod_right</code>	$J(a \mid b) = J(a \mid b \bmod (4 a))$	no



Across several mathematical domains, the theorem reads from the unrestricted argument to the canonical representative.

Especially useful evidence

The character and symbol declarations show that this convention is not peculiar to powers. Their RHS may add casts, products inside a modulus, or **natAbs**; it is still preferred because it identifies the canonical residue data on which the invariant depends.

```

theorem  $\chi_4$ _nat_mod_four (n : Nat) :
   $\chi_4$  n =  $\chi_4$  (n % 4 : Nat)

protected theorem legendreSym.mod (a : Int) :
  legendreSym p a = legendreSym p (a % p)

theorem jacobiSym.mod_right (a : Int) ... :
  J(a | b) = J(a | b % (4 * a.natAbs))
  
```

How the scan was performed

Multiple broad regex families were used, then candidate declarations were inspected manually:

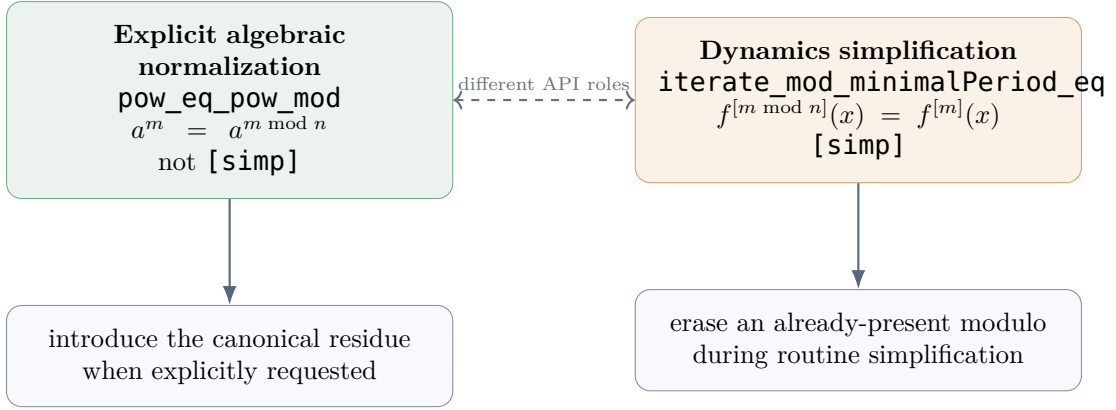
- multiline theorem statements with **mod** in the name and **%** on an equality RHS;
- periodicity families containing **pow_eq_pow_mod**, **iterate_mod**, **mod_left**, or **mod_right**;
- broad equalities whose RHS contains conditionals, compositions, maps, products, or arithmetic structure.

This is a source survey rather than a parser-based census. Exact commands, revisions, file paths, and line references are recorded in **CODEBASE_PATTERN_SURVEY.md**.

Finding: the proposed Frobenius orientation belongs to an established, cross-domain pattern of explicit residue normalization.

Appendix B. Counter-patterns and larger RHSs

A fair survey also finds the reverse direction. The difference clarifies why *simp policy*, not syntax-tree size alone, matters.



The reverse-oriented family

```
theorem IsPeriodicPt.iterate_mod_apply ... :
  f^[m % n] x = f^[m] x

@[simp] theorem iterate_mod_minimalPeriod_eq :
  f^[n % minimalPeriod f x] x = f^[n] x

@[simp] theorem pow_mod_period_smul ... :
  m ^ (n % period m a) * a = m ^ n * a
```

This does not undermine the recommendation. It demonstrates that declarations optimized for simplification can choose a different normal form from explicit normalization wrappers. The Frobenius theorem is a direct specialization of the non-`[simp]` group-theoretic theorem.

Interpretation

Is this theorem global simplifier policy?

No: preserve source orientation and rewrite explicitly

Larger RHSs beyond modulo

Lean core and Mathlib routinely expose a larger but more informative RHS:

```
-- Core arithmetic
Int.add_emod:
  (a + b) % n = (a % n + b % n) % n

Int.fmod_eq_emod:
  fmod a b = a % b +
    if 0 ≤ b ∨ b | a then 0 else b

-- Core computation
Thunk.sizeOf_eq:
  sizeof a = 1 + sizeof a.get

-- Mathlib pointwise computation
CStarMatrix.one_apply:
  (1 : CStarMatrix n n A) i j =
    if i = j then 1 else 0

-- Mathlib structural exposure
Embedding.coe_trans:
  ↑(f.trans g) = ↑g ∘ ↑f
```

These are not direct Frobenius precedents, but they establish the broader design rule: orientation can expose canonical computational, pointwise, or compositional structure even when the printed RHS grows.

Survey-adjusted conclusion

The source tree contains both orientations, selected for different API roles. The closest non-`[simp]` algebraic and number-theoretic precedents use unrestricted input on the left and residue-normalized input on the right. That remains the appropriate design for explicit Frobenius normalization.